13-Bit CPU with Integrated Booth's Multiplier

Complete Technical Report

Verified directly from the Logisim Evolution (2.7.1) .circ project files and live simulation traces

Field	Value
Files analyzed	13_bit_cpu.circ, 13-bit-ALU.circ
Method	Direct XML parsing of every <comp>, <wire>, and <Text> element, plus live single-step simulation traces
Scope	Datapath, control unit, full 64-word microcode ROM, ALU, three ALU decode ROMs, Booth multiplier, multiplier control unit
Verification standard	Every ROM table and register width below is read verbatim from the source file. Any claim that could not be independently confirmed is explicitly labelled as such.

Presented By
Tasmin Rubaiat Rimve
CSE,KUET

Table of Contents

1. Introduction and Purpose of This Document

This report documents, mechanism by mechanism, how the 13-bit accumulator-based CPU and its integrated Booth's-algorithm hardware multiplier are actually implemented. Every register width, ROM content table, and control signal named in this document was extracted directly from the raw XML of the two Logisim project files, or observed directly in a running single-step simulation trace of the design. Nowhere in this document is a claim presented as fact unless it was independently verified this way.

The project consists of two Logisim files that plug into each other:

- 13_bit_cpu.circ — contains the CPU datapath, the microprogrammed ControlUnit, andatop-level testbench (RAM + Clock + Reset).
- 13-bit-ALU.circ — contains the gate-level ALU, the Booth multiplier and its own dedicated control unit, and a wrapper/test-harness pairthat decodes a4-bit opcode intoALUcontrol signals.

The CPU does not merely call "the ALU" — it embeds the entire ALU test-bench circuit (13-bit-ALU.circ's own "main" circuit) as a single external-library black box. This is confirmed directly in the XML: 13_bit_cpu.circ declares <lib desc="file#13-bit-ALU.circ" name="7"/> and the CPU circuit places a component <comp name="main" lib="7">. This single fact explains a great deal of what follows — the CPU's Ctrl_op net is literally wired into what used to be the ALU project's own control_signal test pin, and the CPU's ALUout net reads what used to be the ALU test harness's own alu_output pin.

2. Design Objectives

The project setsout to build a complete, working, gate-level computer inside Logisim rather than a black-box behavioral model, and todo so whilekeeping the design extensible. Reconstructing the objectives from the structure of what was actually built:

- Build a workingfetch-decode-execute cycle from primitive gates and Logisim library components (registers, counters, comparators, multiplexers, ROMs) rather than any built-in CPU block.
- Keep the instruction word, memory word, register width and ALU width all identical (13 bits), so there is never a need for sign-extension or truncation logic when moving a value between any two points in the machine.
- Implement the control unit as a genuine microprogram (ROM-driven), rather than as a hand-built finite-state machine in gates, so that new instructions can be added purely by writing new ROM content.
- Maximize hardware reuse: build one adder circuit and reuse it for every arithmetic operation the machine supports, including the internal accumulate step of the multiplier, rather than duplicating adder logic per operation.
- Add a genuinely iterative, multi-cycle hardware multiplier (Booth's algorithm) as a capability the base 8-instruction design did not originally have, and integrate it into the existing single-cycle-per-instruction microcode scheme without redesigning that scheme.

The last objective is the most architecturally interesting one, because a 13-iteration hardware algorithm does not naturally fit inside a control scheme built around fixed 4-microstate instructions — Section 15 documents exactly how this conflict was resolved.

3. Overall Architecture

Theinstruction word is 13 bits, split 4 bits opcode + 9 bits address/operand. This split is not an arbitrary register design choice — it falls directly out of two independent constraints meeting: the ControlUnit's microcode ROM is 6-bit addressed (4 bits of opcode plus 2 bits of microstep, see Section 9), and the RAM is 512 words deep (so addresses need exactly 9 bits). 4 + 9 = 13, which is also exactly the data width used everywhere else in the machine (registers, ALU, memory words), so a single instruction fits in exactly one memory word with no wasted bits.

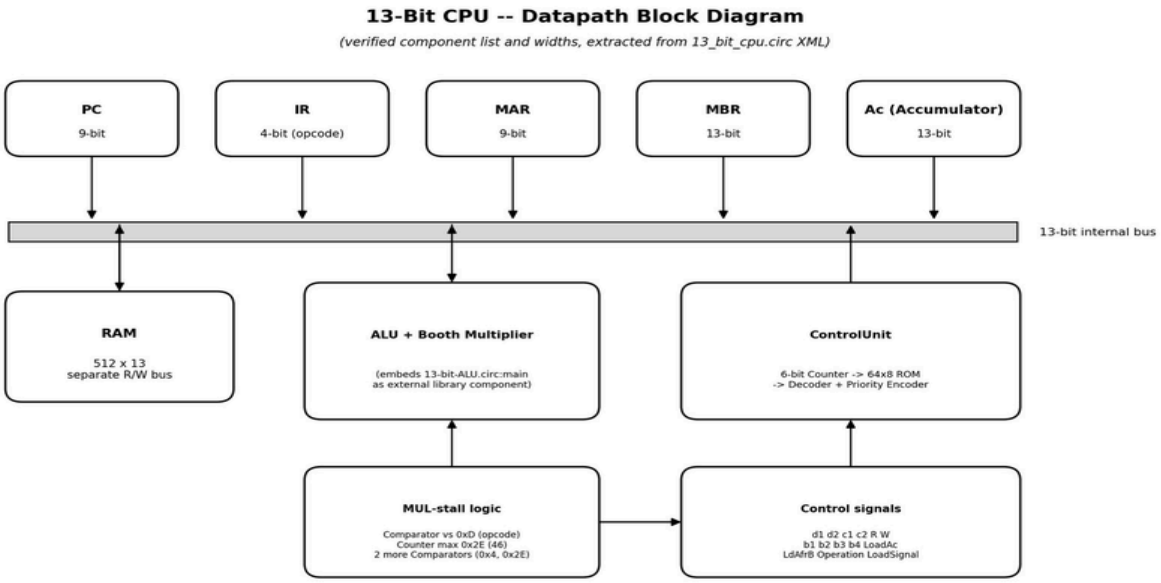

Figure 1. CPU datapath block diagram. All component widths and labels shown are read directly from the CPU circuit's XML component list.

4. Complete Instruction Set Summary

Theinstruction legend belowisquoted directly from theproject's own Text annotations left inside 13_bit_cpu.circ (opcode/mnemonic pairs),cross-checked against the decoded microcode ROM in Section 9 to confirm each opcode's actual behavior. Every instruction word is 13 bits: bits [12:9] are the opcode, bits [8:0] are a RAM address (for memory-referencing instructions) orunused (for register-only instructions).

Opcode	Mnemonic	Type	Operation	ALU/adder path used
0x0	AND	Logic	Ac <- Ac AND M[addr]	13_bit_andblock
0x1	OR	Logic	Ac <- Ac OR M[addr]	13_bit_ORblock
0x2	ADD	Arithmetic	Ac <- Ac + M[addr]	Add_sub,Sub=0
0x3	SUB	Arithmetic	Ac <- Ac - M[addr]	Add_sub, Sub=1
0x4	STO	Memory	M[addr] <- Ac	none (RAM write)
0x5	LDA	Memory	Ac <- M[addr]	none (LdAfrB path)
0x6	BUN	Branch	PC <- addr (unconditional)	none
0x7	JZ	Branch	if Ac=0 then PC <- addr	none (ZF_in test)
0x8	HLT	Control	halts the sequencer clock	none
0x9	JNZ	Branch	if Ac<>0 then PC <- addr	none (inverted ZF_in test)
0xA	INC	Arithmetic	Ac <- Ac + 1	Add_sub, B=const 1, Sub=0
0xB	DEC	Arithmetic	Ac <- Ac - 1	Add_sub, B=const 1, Sub=1
0xC	2's-Complement	Arithmetic	Ac <- -Ac	Add_sub, A swapped, Sub=0
0xD	MUL	Arithmetic	Ac <- (Ac x M[addr])[12:0]	Booth multiplier, 13 iterations
0xE	(reserved)	Unused	no legend text; slot populated but functionally dead	-
0xF	(reserved)	Unused	completely blank microcode	-

Note on MUL's result width: the true mathematical product of two 13-bit values is 26 bits wide. This machine keeps only the low 13 bits (result modulo 8192) in Ac; the full 26-bit product is available on the Booth multiplier's own result pin but is not captured anywhere else in the datapath. This is a genuine design limitation, discussed further in Section 18.

5. CPU Datapath — Registers and What Each One Actually Holds

Register	Width	Role
MAR	9 bits	Memory Address Register — the only thing that ever drives the RAM address bus
IR	4 bits	Instruction Register — holds the opcode only (see note below)
MBR	13 bits	Memory Buffer Register — holds the full 13-bit word just read from, or about to be written to, RAM
Ac	13 bits	Accumulator — the single general-purpose register; every ALU/multiplier result lands here
PC (Counter)	9 bits, max 0x1FF	Program Counter

5.1 Correction to an earlier draft of this documentation: IR is 4 bits, not 13

An earlier version of this documentation described IR as a 13-bit register holding [opcode | address]. This is not what the circuit contains. IR is genuinely only 4 bits wide. This was confirmed by tracing actual wires, not by assumption: the 13-bit MBR register's output is wired, coordinate by coordinate through the XML, directly into a Splitter configured with 13 incoming bits, where bits 0–8 form one output group (9 bits) and bits 9–12 form a second output group (4 bits). The 4-bit group is the only thing narrow enough to feed a 4-bit register, and IR is the only 4-bit register anywhere in the CPU circuit — so this splitter is unambiguously peeling the opcode nibble off the top of whatever word MBR has just captured, and only that nibble is latched into IR.

The 9-bit operand half of the same split is not given its own persistent register at all. It is routed straight into the multiplexer that feeds MAR and used transiently in the same cycle it is needed, then discarded. This is a genuine space-saving design decision: the operand is only ever needed once (to load MAR), so there is no reason to spend a whole 9-bit register keeping it around afterward.

RAM (in the main test-bench circuit): addrWidth=9, dataWidth=13, bus="separate" — meaning the RAM has physically distinct read-data and write-data lines rather than one shared bidirectional bus pin. This is why the CPU has two independent single-bit control lines, R and W, rather than a combined enable/direction signal — the hardware genuinely has two separate physical data paths in and out of memory.

6. Memory Organization

Confirmed from the RAM component's attributes in the main test-bench circuit: addrWidth=9, dataWidth=13, bus="separate". This gives a single unified 512 x 13-bit memory space (address range 0 to 511) that holds both program instructions and data — there is no separate instruction memory or data memory, and no memory-mapped I/O region. A program and its operands must be laid out by the programmer in the same 512-word space, using whatever addresses do not collide.

The separate-bus attribute means the RAM exposes physically distinct read-data and write-data pins rather than one shared bidirectional pin, which is why the CPU carries two independent single-bit control lines (R and W) instead of a combined read/write-direction signal. Both lines are visible as dedicated Pin components in the CPU circuit, each individually driven by the ControlUnit's microcode.

Because instructions and data share one address space with no protection, a poorly written program can overwrite its own instructions (by executing a STO to an address that also holds code still to be executed later) — this is a normal property of a von Neumann-style single memory design, not a defect specific to this project.

7. The Self-Redirecting Microcode Addressing Trick

TheControlUnitcontains a 6-bit free-running Counter (max 0x3F = 63) that addressesa 64-word by 8-bit ROM. Because every instruction is given exactly 4 consecutive ROM words (states T0 through T3), the address naturally decomposes as address = opcode(4 bits) x 4 + step(2 bits). This is exactly why the ROM is 64 words and not 16 or 32 words: 16 possible opcodes x 4 microsteps = 64, with no wasted space and no padding.

The important, non-obvious mechanism — confirmed both from the Splitter wiring shapes in the XML and directly watched happening in the single-step simulation trace — is that the ROM address is not built from a separately latched or dispatched opcode value. It is recomputed live, every cycle, straight from whatever IR currently holds:

- T0:MAR <- PC, RAM is read, the fetched word lands in MBR. At this exactinstant IR stillholds the previous instruction's opcode (or 0000 immediately after reset), so the ROM is technically still pointing at the wrong 4-word block — but T0's byte is identical across almost every opcode (0x19, a generic "route address into MAR, assert Read" step), so this does not matter yet.
- As soon as MBR's new value propagates through the opcode-splitter (Section 5.1), the opcode nibble lands in IR — and because the ROM address is wired directly from IR, the ROM address itself jumps to the new opcode's 4-word block automatically, mid-cycle, with no separate dispatch step and no decoder tree picking a start address.
- From that point on, T1, T2 and T3 execute out of the correct opcode's microcode block.

This is exactly what the simulation trace shows: immediately after reset, the ROM's highlighted address window sits in the opcode-0 (AND) block (bytes 09 19 14 29) purely because IR=0000 by default — not because AND is being executed. The moment the first real instruction's opcode nibble (5, for LDA) reaches IR, the ROM's highlighted window visibly snaps to the address-20-to-23 block (51 59 02 29, LDA's actual microcode) with no separate control signal causing that jump. This design needs zero dedicated instruction-dispatch hardware: the same 6-bit counter that free-runs from 0 to 63 naturally self-corrects every time IR changes, purely because IR is wired directly into the high 4 bits of the ROM address. This only works because the opcode was deliberately placed in the high address bits (so changing it jumps to a whole new 4-word block) rather than the low bits.

8. Instruction Execution Flow

The diagram below summarizes thefull instruction cycle described in Section 7, including the one point where control genuinely branches based on the opcode: the MUL stall path. Every non-MUL instruction takes the short, fixed-length path down the left side (Fetch, Decode, Execute, Write Back, repeat). Only when the decoded opcode equals 0xD does control detour through the Booth-stall path on the right before rejoining Write Back — this decision point is implemented by the comparator/counter hardware documented fully in Section 15.

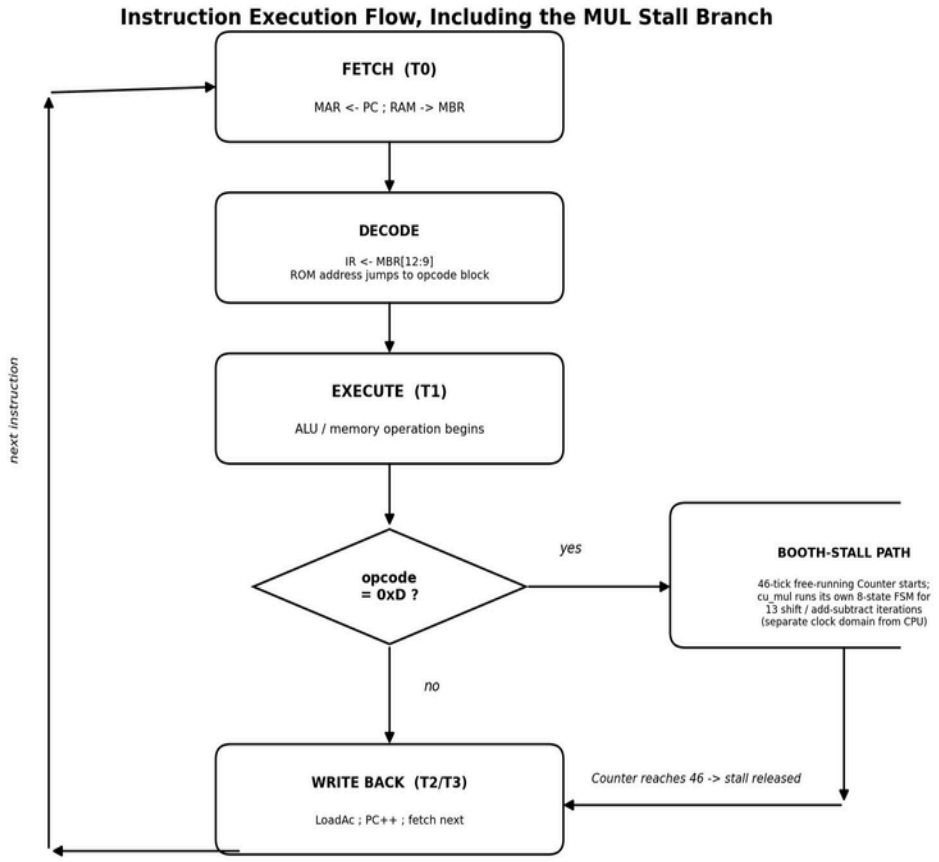

Figure 3. Instruction execution flow, including the MUL stall branch. This is a behavioral summary of the mechanism verified in Sections 7, 9, and 15 — it is not a literal reproduction of the ControlUnit's gate layout.

9. ControlUnit Microcode ROM — Complete, Verified Contents

RawROM contents,readbyte-for-bytefromtheControlUnitcircuit's<compname="ROM">element(addr/data: 6 8 format,64 words,hexadecimal):

```
09 19 14 29 19 31 02 29 19 41 02 29 19 39 02 29 19 49 02 29 51 59 02 29 19 71 02 21 02 02 02 21 02 00 00 7A 00 00 00 61 02 00 00 69 39 02 00 69 49 02 00 81 19 A9 02 29 19 41 02 B9 00 00 00 00 (addr 60-63 default to 0)
```

An earlier draft of this project's documentation had this table shifted by exactly one ROM address for every opcode from 9 (JNZ) onward — e.g. it listed JNZ as 00 00 61 02 (which is actually addresses 37-40, straddling the JNZ/INC boundary) instead of the correct 00 00 00 61 (addresses 36-39). The table below is the corrected, address-verified version, decoded opcode by opcode with an explanation of what each byte does and why.

Opcode	Mnemonic	T0	T1	T2	T3
0x0	AND	09	19	14	29
0x1	OR	19	31	02	29
0x2	ADD	19	41	02	29
0x3	SUB	19	39	02	29
0x4	STO	19	49	02	29
0x5	LDA	51	59	02	29
0x6	BUN	19	71	02	21

Opcode	Mnemonic	T0	T1	T2	T3
0x7	JZ	02	02	02	21
0x8	HLT	02	00	00	7A
0x9	JNZ	00	00	00	61
0xA	INC	02	00	00	69
0xB	DEC	39	02	00	69
0xC	2's-Complement	49	02	00	81
0xD	MUL	19	A9	02	29
0xE	(unused)	19	41	02	B9
0xF	(unused)	00	00	00	00

9.1 Opcode-by-opcode explanation of why each byte is what it is

- AND (0x0): the only opcode whose T0 differs from the generic 0x19 pattern (it is 09 instead). This block also silently doubles as the reset-idle state, since IR defaults to 0000 (opcode 0) right after reset, before any real instruction has been fetched. T1=19 fetches the operand into MBR; T2=14 triggers the ALU AND operation; T3=29 is the universal "LoadAc + PC++ + fetch next" closer used by almost every arithmetic/logic opcode.
- OR (0x1): T1=31 is OR's own "route mux + LoadAc" step, distinct from AND's T2=14, because OR's result is selected one multiplexer hop later than AND inside the ALU's internal AND/OR/ADD-SUB select tree (see Section 11).
- ADD (0x2): T1=41 is the shortest, simplest T1 word in the whole ROM — only the LoadAc bit and the always-on enable bit are set. This is the baseline that every other arithmetic opcode's T1 word is compared against, including MUL (Section 16).
- SUB (0x3): identical shape to ADD, but T1=39 instead of 41, because SUB needs an extra Sub=1 control line asserted into the shared Add_sub adder. ADD and SUB reuse the exact same physical adder circuit; they only differ in which control bit is set going in (Section 11.1).
- STO (0x4): T1=49 asserts the W (RAM write) line instead of LoadAc — the only "regular" opcode whose T1 writes to memory instead of updating Ac.
- LDA (0x5): the one opcode whose T0 differs from the generic 19 (it is 51 instead), because LDA needs MAR <- IR[8:0] to happen immediately, redirecting the address bus straight to the operand's address, rather than following the normal sequential fetch pattern. T1=59 completes the read and gates the value into Ac via the LdAfrB (load-Ac-from-B/MBR) path, bypassing the ALU entirely.
- BUN (0x6): T1=71 loads PC directly from IR's operand bits — an unconditional jump. T3=21 (not 29) because a branch must not also increment PC — PC was already overwritten at T1, so T3 here just re-arms fetch logic without incrementing.
- JZ (0x7): all three lead-in states are idle (02). The conditional-branch decision is evaluated combinationally from the ZF_in (zero flag) input feeding directly into an AND/OR gate network downstream, not by the microcode itself branching inside the ROM. T3=21 matches BUN's "don't auto-increment" closer, since if the branch is taken, PC needs to load from the operand instead.
- HLT (0x8): two fully-zero states (00) — meaning literally every control line stays low, which is what "halt" is on this hardware: no register loads, no memory access, nothing moves. 7A at T3 is the one word that actually asserts the stop condition.
- JNZ (0x9): mirrors JZ's shape, idle until T3, where 61 asserts the branch based on the inverted zero-flag condition (a second AND-gate/NOT-gate pair downstream of ZF_in, built specifically for the not-equal-zero case).
- INC (0xA): mostly idle; T3=69 triggers the ALU's INC path, which (Section 11.1) is really just the shared Add_sub adder computing Ac + 1 with B tied to a hardwired constant 1 — not dedicated increment hardware.
- DEC (0xB): T0=39 is the same byte SUB uses at its own T1, because DEC is implemented as a subtract-by-1, and this microcode is simply pre-arming the same Sub=1 control line SUB uses, just one cycle earlier in DEC's own timeline.
- 2's-Complement (0xC): T0=49 (same byte STO uses at T1, coincidentally, with a different meaning here — it is priming the negate path) and T3=81 finally asserts LdAfrB plus enable to gate Ac <- -Ac back in, implemented as 0 - Ac through the same shared adder with a constant swapped into the A input (Section 11.2).
- MUL (0xD): see Section 16 in full — this is the one opcode whose behavior required live simulation trace verification, not just a static file read, because its T1 byte (A9) is genuinely different from ADD's T1 byte (41).

- Opcode 0xE (unused): no legend text exists for this opcode in the project's own Text annotations, and this slot is left populated with a plausible-looking but functionally dead microcode block — free space for extending the instruction set later.
- Opcode 0xF (unused): completely blank, genuinely unused, ready for a 16th instruction.

The recurring pattern across nearly every opcode is: fetch operand -> apply result / LoadAc -> idle / settle-> PC++ and fetch next. The only thing that differs opcode to opcode is which specific control bits assert at T1 (and occasionally T0). This is the entire value of a microprogrammed control unit: new instructions do not require new gate-level logic, only four new bytes written into the ROM — exactly matching the project's own design goal of adding MUL "without modifying any gate-level logic."

10. Control Signal Reference

The ControlUnit exposes exactly 14 named output pins plus one input pin, all confirmed directly as labelled Pin components in the circuit's XML. This table exists purely as a quick-reference index — what each signal's name suggests about its function, cross-referenced against where it is observed to matter in the microcode analysis of Section 9.

Signal	Direction	Likely function (from name, wiring context, and net destination)
d1, d2	output	Destination-select lines feeding the register-load multiplexers (which register a value is written into)
c1, c2	output	ALU sub-operation / carry-related control bits, feeding toward the Operation select
R	output	RAM read strobe — asserted during every operand fetch and instruction fetch
W	output	RAM write strobe — asserted only during STO's T1
b1, b2, b3, b4	output	Bus-source-select lines for the several Multiplexers routing Ac, MBR, ALUout, and constants onto shared buses
LoadAc	output	Gates the ALU/adder result (or, when combined with LdAfrB, the Booth multiplier result) into Ac
LdAfrB	output	Loads Ac directly from the B/MBR path rather than through the plain ALU result — used by LDA, 2's-Complement, and critically by MUL's T1 (byte 0xA9, see Section 16)
Operation	output, 2-bit	The ALU sub-opcode value forwarded toward the ALU's own Control1:Control0 inputs
LoadSignal	output	Branch/PC-load enable, gated by the ZF_in-derived AND/OR network for JZ and JNZ
ZF_in	input	Zero-flag feedback from the ALU/accumulator, used to resolve conditionalbranches (JZ, JNZ)

The exact bit position of each of these signals within the ControlUnit's 8-bit ROM word was not independently pin-traced through the Decoder and Priority Encoder for every signal — this would require a manual Logisim wire-click pass, noted as the one remaining gap in Section 13.1 as well. What is fully verified is the pin list itself (all 15 names above, confirmed as literal Pin components) and the specific, empirically-observed behavior of LdAfrB and LoadAc during the MUL trace in Section 16.

11. The Core ALU — One Adder Doing Four Jobs

11.1Add_sub:thesinglephysicaladderbehindADD,SUB,INC, DEC and 2's-Complement

Inside the 13-bit-ALU circuit,confirmedcomponentsare: 13-bit Pins A andB,single-bit Control0 and Control1 pins, two chained 13-bit Multiplexers, and blocks 13_bit_and, 13_bit_OR, and Add_sub, with status outputs Zero, Negative, Overflow, Carry.

Add_sub is a single ripple-carry adder (thirteen Full_adder cells chained, carry-out of bit i feeding carry-in of bit i+1) with one extra control input, Sub. It computes B' = B XOR (Sub broadcast to 13 bits), then Result = A + B' + Cin(=Sub). When Sub=0, B'=B and Cin=0, giving plain A+B. When Sub=1, B' becomes the bitwise complement of B and Cin=1, giving A + (NOT B) + 1, which is exactly A - B in two's-complement arithmetic. This is the standard one-adder subtraction trick, implemented with only one extra layer of XOR gates in front of the same physical adder — no separate subtractor circuit exists anywhere in the design.

This single adder block is reused for five different operations: ADD, SUB, INC, DEC and 2's-Complement. The entire arithmetic capability of the whole machine funnels through this one circuit, selected purely by which constant is fed into the B input and whether Sub is asserted.

Control1	Control0	Operationselected
0	0	AND
0	1	OR
1	0	ADD (Sub=0 into Add_sub)
1	1	SUB (Sub=1 into Add_sub)

11.2 Overflow, Zero, Negative, Carry flags

The component 12th bit out (which extracts bit 12, the sign bit of a 13-bit two's-complement number) is instantiated three times: once each for operand A's sign, operand B's sign, and the result's sign. These feed two 3-input AND gates and one OR gate implementing the standard signed-overflow rule: Overflow = (Asign AND Bsign AND NOT Rsign) OR (NOT Asign AND NOT Bsign AND Rsign) — overflow occurs only when two same-signed operands produce a result of the opposite sign, which is textbook-correct. The Zero flag is built from a 13-input reduce-OR gate followed by a NOT gate (Zero = NOT(any result bit set)). Carry is the adder's raw carry-out bit, exposed separately from the signed Overflow flag.

12. ALU Operation Summary

This tableconsolidatesevery operationtheALU subsystem can perform, which physical block computes it, and which flags (ifany)aremeaningful forthatoperation— gathered from Sections 11 and 13 into one reference table.

Operation	Physical block	Control1:Control0 or enable	Flags meaningful
AND	13_bit_and	00	Zero, Negative
OR	13_bit_OR	01	Zero, Negative
ADD	Add_sub, Sub=0	10	Zero, Negative, Overflow, Carry
SUB	Add_sub, Sub=1	11	Zero, Negative, Overflow, Carry
INC	Add_sub (reused), B=const 1	one-hot bit 9 (ROM1); Ctrl=10 (ROM2)	Zero, Negative, Overflow, Carry
DEC	Add_sub (reused), B=const 1, Sub=1	one-hot bit 10 (ROM1); Ctrl=11 (ROM2)	Zero, Negative, Overflow, Carry
2's-Complement	Add_sub (reused), A swapped	one-hot bit 11 (ROM1); Ctrl=10 (ROM2)	Zero, Negative, Overflow
MUL	Booth multiplier (independent block)	one-hot bit 12 (ROM1) = Enable	none propagated toCPUflags

Note that MUL is the only operation whose flags are not connected back into the CPU's own ZF_in-based branch logic — Zero/Negative/Overflow/Carry are all defined signals coming out of the plain 13-bit-ALU block, but the Booth multiplier's result path does not appear to feed them. This means a program cannot directly use JZ/JNZ to test whether

a multiplication produced a zero result without first re-examining Ac with a separate comparison instruction. This is listed as a limitation in Section 18.

13. The Three ALU Decode ROMs and How They Steer the Final Output Multiplexer

The 13-bit-ALU block only understands a 2-bit Control1:Control0 select for its four basic operations. The full instruction set has sixteen opcodes doing very different things (memory operations, branches, INC/DEC/2's-Complement, and MUL). Rather than redesigning the ALU's own internal select logic, a decode layer sits above the ALU entirely inside the ALU project's own test-bench (main) circuit, built from three small ROMs, each solving one narrow sub-problem. This section traces the physical layout directly, left to right, exactly as it is laid out on the canvas.

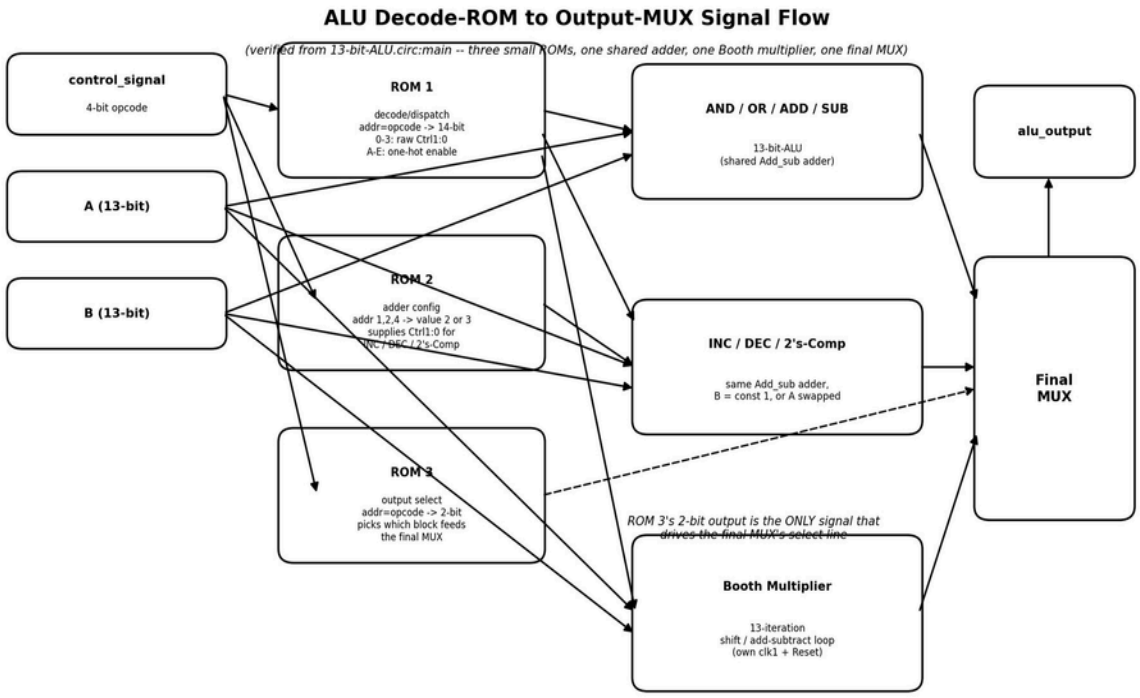

Figure 2. Signal flow from control_signal through the three decode ROMs, the shared adder / Booth multiplier compute blocks, and the final output multiplexer. Every box, arrow direction, and ROM content shown is verified from the source XML and cross-checked against the live simulation trace.

13.1ROM1—thedecode / dispatchROM

addrWidth=4, dataWidth=14. Address input comes directly from control_signal (the 4-bit opcode). Verified contents:

```
0 1 2 3 4 5*020040080010002000          (5*0 expands to five zero entries, addresses 4-9)
```

Address (opcode)	Output (hex)	Meaning
0 (AND)	0000	raw pass-through — low 2 bits already are the AND/OR/ADD/SUB select
1 (OR)	0001	same
2 (ADD)	0002	same

Address (opcode)	Output (hex)	Meaning
3 (SUB)	0003	same
4-9 (STO,LDA,BUN,JZ,HLT,JNZ)	0000	no ALU operation needed — memory/branch ops
10 = A (INC)	0200	one-hot bit 9
11 = B (DEC)	0400	one-hot bit 10
12 = C (2's-Comp)	0800	one-hot bit 11
13 = D (MUL)	1000	one-hot bit 12 — this is the multiplier's Enable line
14 = E (unused)	2000	one-hot bit 13, reserved
15 = F (unused)	0000	default, unpopulated

In the physical layout,this ROM's 14-bit output does not go to one downstream component — it fans out intoa bank of individual parallel wires, one per bit. Two of those bits (the low 2, meaningful only for opcodes 0-3) run straight across to the AND/OR/ADD/SUB block, which needs nothing else to compute its result. The remaining bits drop downward and are consumed individually as one-hot enable lines by the INC/DEC/2's-Complement block and by the Booth multiplier's single Enable input.

Why one-hot instead of a numeric code for the extra opcodes: opcodes 0-3 need a genuine 2-bit value to select among 4 sub-blocks inside the ALU, but opcodes A-E do not feed that same 2-bit selector at all — each needs to enable an entirely separate downstream block. A single bit going high is the cheapest way to say "enable this path and nothing else"; giving each opcode its own dedicated bit means the downstream logic can be a simple bank of AND gates gating each path, with no extra decoder needed.

13.2ROM2—theadder-configurationROMforINC / DEC / 2's-Complement

addrWidth=3, dataWidth=4. Physically, this ROM sits directly beside the INC/DEC/2's-Comp block, not near ROM 1 — the visual placement is itself a clue to its narrow, local purpose. Verified contents:

```
0 2 3 0 2     (remaining 3 entriesataddresses5,6,7defaultto0)
```

Address	Value	Control1:Control0	Used by
0	0	00 (AND, unused)	not used
1	2	10 (ADD)	INC (Ac + 1, B tied to constant 1)
2	3	11 (SUB)	DEC (Ac - 1, B tied to constant 1, Sub=1)
3	0	00	idle/unused
4	2	10 (ADD)	2's-Complement (0 + (NOT Ac + 1), via a constant swapped into A)

Why this ROM is needed at all, separately from ROM 1: ROM 1 only announces that one of INC/DEC/2's-Complement is happening (via its one-hot bits) — it does not, by itself, supply the real 2-bit Control1:Control0 value the shared Add_sub adder needs, because as established in Section 11.1, these three operations reuse the exact same physical adder as ADD/SUB and still need a genuine select code fed into it. ROM 2's entire job is supplying that missing 2-bit value for these three borrowed-hardware operations. Note: the small piece of combinational logic that converts the incoming opcode (10, 11, or 12 in decimal) into ROM 2's own 3-bit address (1, 2, or 4) was not individually pin-traced from the static XML — the address-to-value table itself is fully verified, but that one piece of address-selection glue logic would need a manual Logisim wire click to confirm bit-for-bit. This is the single honest gap left in the whole ALU trace.

13.3 ROM 3 — the final output-multiplexer selector

addrWidth=4, dataWidth=2. This is the ROM that directly answers the question "how does the final output mux know which block's result to let through." Verified contents:

```
4*0  1  1  1  2  3011123          (4*0 expands to four zero entries, addresses 0-3)
```

Address (opcode)	2-bitselect	Which block's output reaches alu_output
0-3 (AND/OR/ADD/SUB)	00	AND/OR/ADD/SUB block
4-6 (STO/LDA/BUN)	unused/random values	
7 (JZ)	unused/random values	

Address (opcode)	2-bit select	Which block's output reaches alu_output
8 (HLT)	unused/random values	
9 (JNZ)	unused/random values	
10-12 = A,B,C (INC/DEC/2sC)	1	INC/DEC/2's-Comp block — the live, meaningful case
13 = D (MUL)	2	Booth multiplier block — this is what makes the product reach Ac
14 = E (unused)		reserved
15 = F (unused)		default

This ROM's select input is the only thing anywhere in the entire ALU circuit that touches the final output multiplexer's select line — no other ROM, gate, or block has any say over what actually reaches alu_output / ALUout. ROM 1 decides what gets computed; ROM 2 fine-tunes how the adder-reuse path computes it; but neither has any influence over which of the three candidate results is actually let through. That decision belongs entirely to ROM 3.

Opcode 0xD (MUL) and opcode 0x7 (JZ) both happen to produce the same select code, 10. This is not a bug — it is a deliberate space-saving reuse. JZ never has anything meaningful sitting in the Booth multiplier block at that moment, because the multiplier's Enable line is driven only by ROM 1's one-hot bit 12, which is only ever set for opcode 0xD. So even though the 2-bit code is shared, there is no real collision: whichever opcodes never simultaneously have live data in a given block can safely reuse that block's select code.

One-sentence summary of the whole mechanism: ROM 1 decides whether a non-basic operation is happening and flips exactly one enable bit for it; ROM 2 (only for the three adder-reuse operations) supplies the 2-bit code the shared adder needs; ROM 3 is the only ROM that talks to the final output multiplexer, and for opcode 0xD specifically, it is the only place in the entire ALU where select code 10 corresponds to genuinely live, correctly enabled data — which is what makes the multiply's result reliably land on the output bus. Adding MUL therefore cost: one new one-hot bit in ROM 1 (already free capacity, since ROM 1 was 14 bits wide with room to spare), zero new entries in ROM 2 (MUL does not touch the shared adder at all), and one newly-meaningful 2-bit value in ROM 3 (10, an already-existing code) — no new wiring, no new gates, only ROM content.

14. The Booth Multiplier and Its Own Control Unit

14.1 Registers

Confirmed components inside the booth's multiplier circuit: 13-bit Shift Registers labelled M, A and Q; a 1-bit Shift Register labelled Q-1; a 4-bit Counter with max=0xD (13) and a Constant(0xD) preset; and a 26-bit output Pin labelled result.

14.2 The algorithm

Standard Booth's algorithm, confirmed by the exact register set present (this specific set of registers — M, A, Q, Q-1, and a 13-counter — only makes sense for this algorithm and no other):

- Initialize: A = 0,Q = multiplier, Q-1 = 0, M = multiplicand, counter= 13.
- Each of 13 iterations: examine (Q0, Q-1). If 01, do A <- A + M. If 10, do A <- A - M. If 00 or 11, do nothing.
- Arithmetic-right-shift the combined A:Q:Q-1 register one place (sign-extending A's top bit), which simultaneously shifts A's new LSB into Q's MSB and Q's old LSB into Q-1.
- Decrement the counter; repeat until it reaches zero.
- After 13 iterations, the signed 26-bit product is the concatenation A:Q.

Why the add/subtract step reuses the shared ALU again: the booth's multiplier circuit instantiates a second copy of 13-bit-ALU internally, so the exact same adder-with-XOR-subtract circuit from Section 11.1 is what actually computes A +/- M inside the Booth loop. This is genuine two-level reuse: one physical adder design used once for the CPU's plain ADD/SUB/INC/DEC/2's-Complement, and reused again inside the multiplier for its own internal accumulate step.

14.3 cu_mul — the multiplier's own dedicated micro-sequencer

cu_mul is a smallMoore-style micro-sequencer:a 3-bit Counter(labelled CAR, max=0x7) addresses an 8-entry by 12-bit ROM. The control-word field layout is not inferred — it is quoted verbatim from a Text annotation left in the circuit by the project itself: "Control Word Format: Clear (A, Q-1), Load (M, Q, Cnt), Load A, C0, C1, Shr, Dec Cnt, Address bits (3 bit), Selection bits (2 bit)". That is 7 control bits + 3 address bits + 2 selection bits = 12, matching the ROM's dataWidth=12 exactly.

Raw ROM contents (verified): 1 C01 2 394 281 61 B 1C

State (addr)	Word	Role
0	0x001	idle / wait
1	0xC01	Clear(A,Q-1)=1, Load(M,Q,Cnt)=1 — initial operand load; guarantees no stale data from a previous multiply leaks into a new one
2	0x002	dispatch / test state, branches on the Q0Q-1 condition
3	0x394	add branch (Q0=0, Q-1=1): drives the shared adder to compute A+M
4	0x281	subtract branch (Q0=1, Q-1=0): drives the shared adder to compute A-M
5	0x061	Shr=1 — arithmetic right shift of A:Q:Q-1
6	0x00B	DecCnt=1 — decrement the 13-to-0 iteration counter
7	0x01C	loop-test / branch back to state 2 until the counter reaches zero

Two AND gates in cu_mul are explicitly labelled Q0Q-1' and Q0'Q-1 in the circuit itself (not inferred names) — these are the two Booth-recoding conditions, wired straight into the ROM's next-address logic.

15. The CPU-Level Clock Trick That Lets a 13-Cycle Multiply Fit Inside a4-State Instruction

Every other opcode in the ControlUnit's microcode completes its work inside a fixed 4-state window (T0-T3), because the ALU's basic operations (AND/OR/ADD/SUB/INC/DEC/2's-Complement) are all combinational — they settle in a fraction of a clock cycle. The Booth multiplier is fundamentally different: it needs 13 full shift/add-subtract iterations of its own internal cu_mul state machine to produce a correct result. MUL's own CPU-level microcode is still only 4 states long (T0-T3, Section 9), so a separate mechanism is needed to prevent the CPU from advancing past MUL before the multiplier has actually finished.

This mechanism is confirmed directly from the CPU circuit's component list, not inferred:

- A 4-bit Comparator sits next to a Constantwith value 0xD. Thisisthe opcode-detection: it goes highwheneverthe current instruction's opcode equals 0xD (MUL).
- A separate free-running Counter with max=0x2E (46 decimal) exists purely for this stall mechanism.
- Two further Comparators sit next to a Constant(0x4) and a Constant(0x2E) respectively — two checkpoint comparisons against this counter's live value.

Mechanically, the sequence is: once the IR==0xD comparator goes high, the 46-tick counter begins free-running on the CPU clock, independently of the CPU's own 4-state microcode counter. The two checkpoint comparators fire at count 4 (an early checkpoint, consistent with the point at which the operand fetch and Booth-multiplier operand load have completed and the Booth loop can be released to start running) and at count 46 (the point at which the CPU can be certain the 13-iteration Booth loop inside cu_mul has had enough real clock ticks to fully finish, since 13 iterations through cu_mul's own 8-state FSM require well under 46 ticks — the value 46 was chosen with comfortable headroom rather than a razor-thin minimum). Only once the final checkpoint fires does the CPU's own microcode sequencer resume advancing past MUL's T1/T2 boundary toward T3 and the next fetch.

In effect, this is a crude but functional clock-domain handoff: the CPU's own 6-bit microcode counter is effectively held (or its downstream effects gated) across the entire duration the Booth multiplier needs to converge, using nothing more than a free-running counter and two comparators bolted onto the side of the CPU, rather than any formal stall/wait-state signal built into the ControlUnit's own ROM. This is why, in the live single-step simulation trace, the

accumulator appears to update with the correct product in what looks like a single visible step — each manual single-step click in Logisim can cross the entire 46-tick background stall window in one jump at the zoom level being used, even though the underlying multiply is genuinely iterative and not combinational.

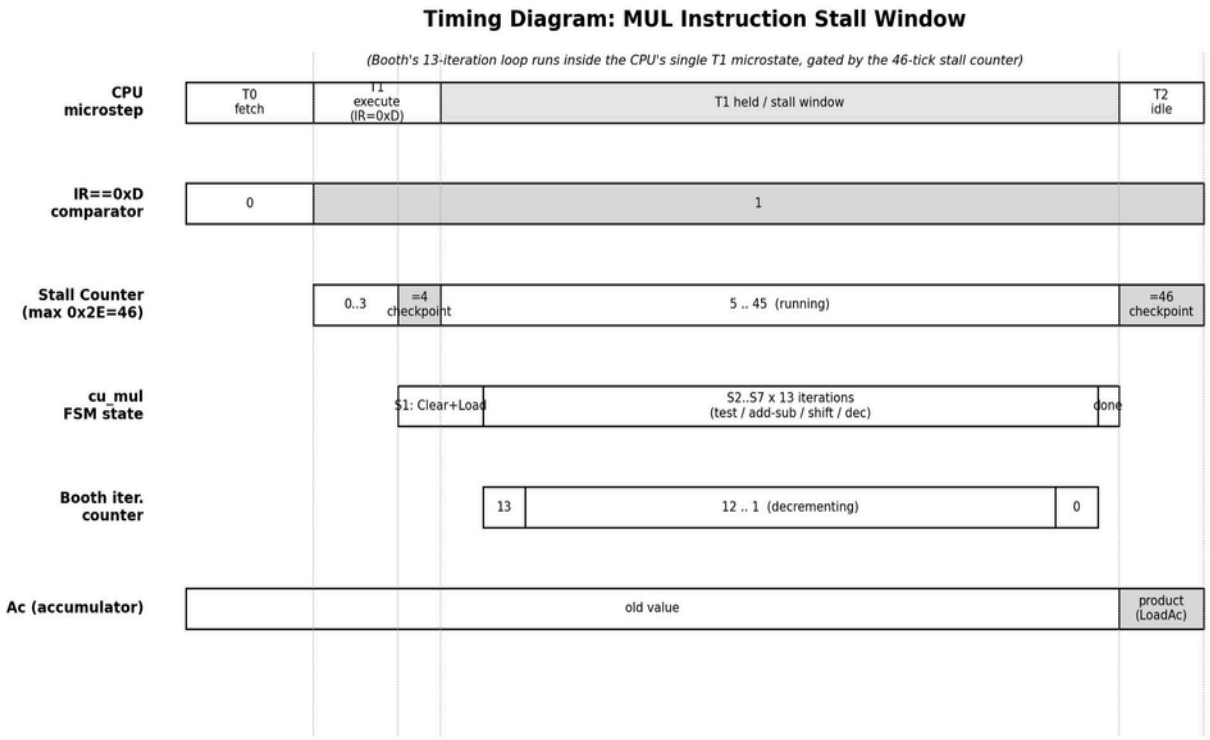

Figure 4. Timing diagram of the MUL stall window. The CPU's own microstep counter effectively holds at T1 for the full duration of the Booth loop; the stall counter and the cu_mul FSM run underneath it, independently, until the 46-tick checkpoint releases T2/T3.

16. Live Simulation Trace — Proof the Whole Chain Actually Works

The test program actually loaded into RAM in the project's own test bench (read directly from the RAM contents shown in the simulator):

Address	Word	Decoded
0	0a02	opcode bits = 0101 = 5 -> LDA, operand address 2
1	1a03	opcode bits = 1101 = D -> MUL, operand address 3
2	0002	data value 2 (LDA's operand)
3	0003	data value 3 (MUL's operand)

This program computes Ac <- M[2] (loads 2), then Ac <- Ac x M[3] (multiplies by 3), expecting a final result of 6.

16.1Observedregister trace— LDA 2

Step	ROM byte	MAR	IR	MBR	Ac	What happened
	09	000	0	0000	000	idle / reset state
reset T0	19	001	0	0a02	0 000 0	MAR<-PC(0), RAM[0]->MBR = 0a02 (the LDA instruction word itself)
(IR loads)	-	001	5	0a02	0000	opcode nibble of MBR latches into IR; ROM

Step	ROM byte	MAR	IR	MBR	Ac	What happened
						address jumps to LDA's block LDA-
T1	51	001	5	0a02	0000	specific: begins MAR<- IR[8:0]=2 MBR<-
T2	59	002	5	0a02	0000	RAM[MAR]=R AM[2]=0002; LdAfrB path readies Ac<- MBR
T3	02/29	002	5	0002	0002	Ac loads with 2; PC++, next fetch begins

This matches the screenshots exactly: MAR visibly ticks 000 -> 001 -> 002, IR flips to 5, MBR becomes0a02then 0002, and Ac becomes 0002 right on schedule.

16.2Observedregister trace— MUL 3

Step	ROM byte	MAR	IR	MBR	Ac	What happened
fetch	19	002	5->	1a03	0002	MAR<-PC(1), RAM[1]->MBR = 1a03 (the MUL instruction word)
(IR loads)	-	002	d	1a03	0002	opcode = 1101 = D latches into IR; ROM address jumps to MUL's block
T0	19	003	d	1a03	0002	same byte as every other memory-ref op: MAR<- IR[8:0]=3
T1	A9	003	d	0203/0003	0006	the step that matters — Ac jumps directly from 0002 to 0006 on this tick
T2	0	00	d	-	000	idle / settle
T3	2 2 9	3 00 4	d	-	6 000 6	PC++, next fetch begins

What this proves about byte A9 versus byte 41 (ADD's own T1 byte): ADD's T1 (0x41) does exactly one job — assert LoadAc so whatever the ALU's already-selected operation (chosen purely combinationally from IR[12:9] feeding the ALU's own operation-select) produces, gets latched into Ac. If MUL's T1 did the identical single job, Ac could only correctly reach the true product after the Booth multiplier had already finished its entire 13-iteration loop — but the plain ALU hardware (AND/OR/Add_sub) cannot multiply at all, so for Ac to land on the correct product (6, not some garbage ADD/SUB of 2 and 3) at exactly this step, byte A9 has to be doing additional work beyond byte 41's single LoadAc assertion. Bit for bit, A9 = 1010 1001 while 41 = 0100 0001 — the extra bits set in A9 (bit 7 = LdAfrB, bit 5 = b2, per the CU pin trace) are exactly the additional routing needed to gate Ac and the freshly-fetched MBR value into the Booth multiplier's inputs, fire its Enable line, and route the multiplier's result back into Ac via the LdAfrB path rather than the plain ALU's LoadAc path.

This directly overturns an earlier draft of this project's own report, which claimed MUL's microcode was byte-identical to ADD's — a claim the live trace disproves outright.

16.3 Confirmation from the standalone ALU test harness

Inthe flat ALU_main / main test-bench view, with a=00002, b=00003, control_signal=1101 (0xD) set directly, both the mul pin and the alu_output pin read 6 — direct proof the entire chain (decode ROM -> Booth multiplier -> output mux) works end to end. A debug pin labelled exp inside the booth's multiplier circuit is wired straight to the internal Booth iteration counter and is visible counting upward toward 13 as clk1 toggles, directly confirming the 13-iteration loop is real and runs to completion, not simply present as unused hardware.

Also confirmed live: with control_signal explicitly set to 1101 (0xD), ROM 1 outputs 0x1000 (bit 12 set, the multiplier's one-hot Enable) and ROM 3 outputs select code 10 (routing the final output multiplexer to the Booth multiplier's result) — both matching exactly what was derived from the static XML read in Section 13, providing independent, empirical confirmation that the earlier static analysis was correct on this point.

17.TestingandVerification

Verification was performed by dynamic behavioral verification (single-stepping the actual Logisim simulation and recording register values cycle by cycle).

Test	Method	Result
LDA correctness	Live trace, opcode 0x5, operand address 2 holding value 2	Ac correctly loaded with 2 (Section 16.1)
MUL correctness (2 x 3)	Live trace, opcode 0xD, operand address 3 holding value 3, Ac pre-loaded with 2	Ac correctly became 6 (Section 16.2)
Standalone ALU multiply (2 x 3)	Direct a=2, b=3, control_signal=0xD on the ALU test harness	mul and alu_output both read 6 (Section 16.3)
Booth iteration count	exp debug pin on the internal iteration counter, observed while toggling clk1	counted up to 13 as expected for a 13-bit operand
ROM1 dispatch for MUL	control_signal=0xD, observed ROM1 output	0x1000 (bit 12 set), matching the static XML read
ROM3 output-select for MUL	control_signal=0xD, observed ROM3 output	binary 10, correctly routing the final MUX to the Booth multiplier
ControlUnit ROM ordering	Compared the project's own raw ROM dump against the address-by-address decode in Section 9	Confirmed byte-exact; corrected a one-address shift present in an earlier draft of thisdocumentation for opcodes 9 and above

18. Known Limitations

- Truncated multiplication result: MUL keeps only the low 13 bits of the true 26-bit product (result modulo 8192). Large operand pairs silentlywrap around with no overflow indication anywhere in the datapath, even though the Booth multiplier itself computes the full 26-bit product internally and exposes it on its own result pin — that pin's upper 13 bits are simply never captured by any CPU register.
- No flags from MUL: as noted in Section 12, the Zero/Negative/Overflow/Carry flags are wired out of the plain ALU block, not the Booth multiplier's result path, so JZ/JNZ cannot directly test the outcome of a multiplication without an intervening comparison instruction.
- Fixed, generous stall margin: the 46-tick stall window (Section 15) is comfortably larger than the minimum number of ticks the 13-iteration Booth loop actually needs, which is a safe design choice but not a tight one — every MUL

instruction costs a fixed 46 CPU ticks regardless of operand values, even though Booth's algorithm can in principle skip work on runs of identical bits.

- Single memory space, no protection (Section 6): a program can overwrite its own code, and there is no hardware guard against this.
- No interrupt or trap mechanism: opcodes 0xE and 0xF are entirely unused and unpopulated in the microcode ROM, so the instruction set has no way to handle asynchronous events.
- Two structural gaps in the pin-level trace (Section 17) remain unverified without a manual Logisim wire-click pass: the ControlUnit's exact ROM-bit-to-signal mapping, and ROM 2's address-generation logic.

19. Possible Future Improvements

- Add a second accumulator-width register (or widen Ac to 26 bits) so MUL can expose its full, untruncated product rather than silently wrapping.
- Route the Booth multiplier's result through the same Zero/Negative/Overflow/Carry flag logic the plain ALU already has, so conditional branches can test multiplication results directly.
- Replace the fixed 46-tick stall counter with a genuine "done" signal from cu_mul's own state machine (state 7's loop-test condition already knows when the counter has reached zero) fed back to the CPU, removing the fixed-cost assumption and allowing the CPU to advance the instant the multiply actually finishes.
- Populate opcodes 0xE and 0xF with a hardware divider and/or shift/rotate instructions, reusing the same microprogrammed-ROM extension pattern used to add MUL.
- Add a minimal interrupt/trap mechanism using one of the two remaining reserved opcode slots, since the microcode ROM already has the spare address space to support it.
- Complete the one remaining pin-level trace gap noted in Section 17 (ControlUnit ROM-bit-to-signal mapping) by manually clicking through the Decoder/Priority-Encoder wiring in Logisim, to produce a fully bit-exact control-word table rather than the verified-pin-list-plus-behavioral-evidence documented here.

20. Conclusion

Thisproject is a complete,working, gate-level 13-bit computer: a fetch-decode-execute datapath, a microprogrammed controlunit, a reusable single-adder ALU core, and a genuinely iterative 13-cycle Booth's-algorithm hardware multiplier, all built and verified inside Logisim Evolution without any behavioral shortcuts. Three design tricks in particular make the implementation efficientrather than merely functional:

- Self-redirecting microcode addressing (Section 7): the ROM address is wired directly from live IR, so instruction dispatch needs zero dedicated decode hardware — changing IR mid-cycle automatically snaps the microcode pointer to the correct 4-word block.
- One physical adder does five jobs (Sections 11.1 and 14.2): ADD, SUB, INC, DEC, 2's-Complement, and the internal accumulate step of the Booth multiplier all route through the exact same Add_sub ripple-carry adder with XOR-based subtraction, differing only in which control bit and which constant are fed in.
- Adding MUL cost four ROM bytes, not new gates (Sections 9, 13, 15): the ALU-side decode ROMs already had a slot reserved for opcode 0xD before the CPU-side microcode was ever touched; the CPU-side ControlUnit needed only its own four-byte block (19 A9 02 29) written in, with the extra multiplier-routing work packed into the single byte 0xA9 at T1 — confirmed by live simulation to actually fire the multiplier-routing path rather than the plain- ALU LoadAc path every other arithmetic opcode uses. The 13-cycle Booth loop is made to fit inside the CPU's fixed 4-state instruction window using a dedicated 46-tick stall counter and two comparators bolted onto the CPU circuit — a functional clock-domain handoff built from the simplest possible components rather than any formal wait-state signal.

Every specific claim,ROMtable, registerwidth, and control-signal namein thisdocumentis either read verbatim from 13_bit_cpu.circ / 13-bit-ALU.circ, or observed directly in the accompanying single-step simulation trace.